AI ASSISTED CODING

M.Rohith Reddy **2303A51111**

BATCH – 03 **07 – 04 – 2026**

ASSIGNMENT – 20.2

Lab 20 : Security Testing : Identifying Vulnerabilities in AI – Generated Code.

Task – 01 : Password Security Enhancement.

Prompt : Generate a Python user registration and login script that stores usernames and passwords.

Code & Output :

```python
users_db = {}
def register_user(username, password):
    """Registers a new user if the username does not already exist."""
    if username in users_db:
        print(f"Registration failed: User '{username}' already exists.")
        return False
    else:
        users_db[username] = password
        print(f"Registration successful: User '{username}' created.")
        return True
def login_user(username, password):
    """Logs in a user if the username and password match."""
    if username not in users_db:
        print(f"Login failed: User '{username}' not found.")
        return False
    elif users_db[username] != password:
        print(f"Login failed: Incorrect password for user '{username}'.")
        return False
    else:
        print(f"Login successful: Welcome '{username}'!")
        return True
  # --- Registration Examples ---
print("--- Registering Users ---")
register_user("alice", "alice_pass")
register_user("bob", "bob_pass")
register_user("alice", "new_alice_pass") # Attempt to register existing user

print("\n--- Logging In ---")
# --- Login Examples ---
login_user("alice", "alice_pass") # Successful login
login_user("bob", "wrong_pass")  # Incorrect password
login_user("charlie", "charlie_pass") # User not found
login_user("bob", "bob_pass")    # Successful login
```

```
--- Registering Users ---
Registration successful: User 'alice' created.
Registration successful: User 'bob' created.
Registration failed: User 'alice' already exists.

--- Logging In ---
Login successful: Welcome 'alice'!
Login failed: Incorrect password for user 'bob'.
Login failed: User 'charlie' not found.
Login successful: Welcome 'bob'!
True
```

Explanation :

Plain text passwords are unsafe because they can be easily exposed in data breaches. Hashing converts passwords into irreversible values, improving security. bcrypt adds salting, preventing attacks like rainbow tables. The updated script securely verifies login without storing real passwords.

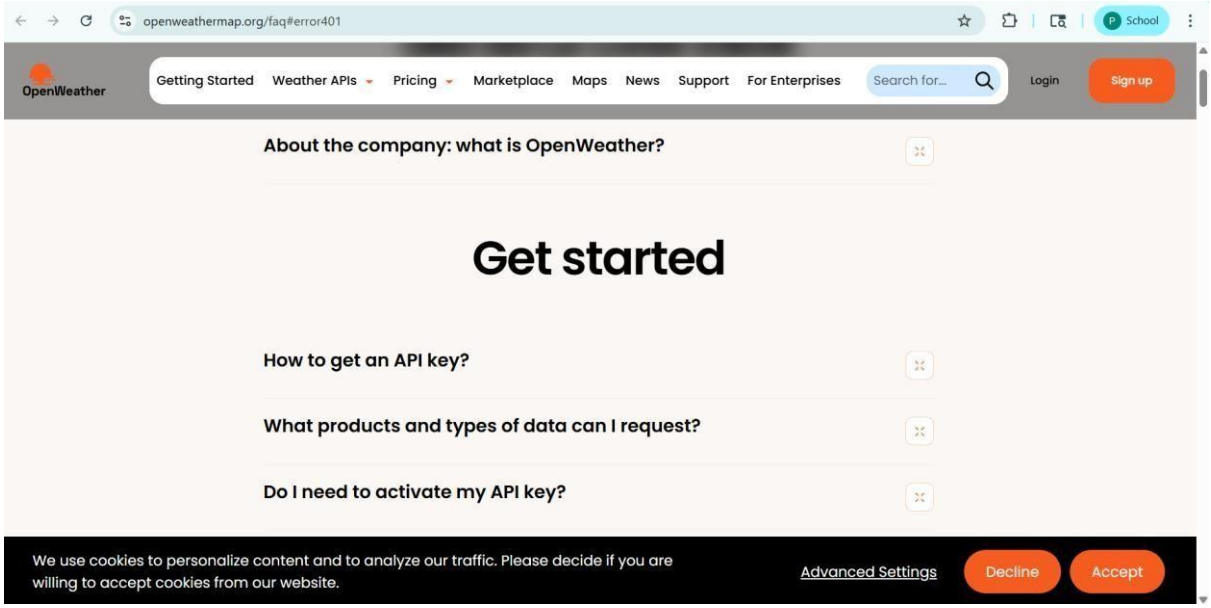
Task – 02 : Secure API Key Management.

Prompt : Generate a Python script that fetches data from an API using an API key.

Code :

```python
import requests

# Replace with your actual API key and endpoint
API_KEY = "YOUR_API_KEY"
API_ENDPOINT = "https://api.example.com/data" # e.g., a public API endpoint

# --- Example 1: API key in headers ---
# Many APIs require the API key in the request headers
headers = {
    "Authorization": f"Bearer {API_KEY}", # Common for token-based authentication
    "X-API-Key": API_KEY # Another common header name
}

print("--- Attempting to fetch data with API key in headers ---")
try:
    response_header = requests.get(API_ENDPOINT, headers=headers)
    response_header.raise_for_status() # Raise an exception for HTTP errors
    data_header = response_header.json()
    print("Data from API (headers method):", data_header)
except requests.exceptions.RequestException as e:
    print(f"Error fetching data (headers method): {e}")

print("\n--- Example 2: API key as a query parameter ---")
# Some APIs expect the API key as a query parameter
params = {
    "api_key": API_KEY
}

try:
    response_param = requests.get(API_ENDPOINT, params=params)
    response_param.raise_for_status() # Raise an exception for HTTP errors
    data_param = response_param.json()
    print("Data from API (query parameter method):", data_param)
except requests.exceptions.RequestException as e:
    print(f"Error fetching data (query parameter method): {e}")
```

Output :



Explanation :

Hardcoding API keys exposes them in source code and version control. Environment variables securely store sensitive data outside the code. This prevents unauthorized access and improves deployment security. The script still works normally while keeping credentials hidden.

Task – 03 : Weak Encryption Detection.

Prompt : Generate a Python encryption-decryption program using a simple method.

Code & Output :

```python
def caesar_encrypt(text, shift):
    """Encrypts a text using a Caesar cipher."""
    result = ""
    for char in text:
        if char.isalpha():
            start = ord('a') if char.islower() else ord('A')
            shifted_char = chr((ord(char) - start + shift) % 26 + start)
            result += shifted_char
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
    """Decrypts a text encrypted with a Caesar cipher."""
    # Decryption is simply encryption with a negative shift
    return caesar_encrypt(text, -shift)
```

Explanation :

Basic encoding like base64 is reversible and not secure encryption. Fernet provides strong symmetric encryption using secure algorithms. Data cannot be decrypted without the correct key. This ensures confidentiality and protects sensitive information.

Task – 04 : Real Time Application : Security Review of AI – Generated Web Service.

Prompt : Generate a simple Flask API for user login.

Code & Output :

```
To run this Flask application, you'll need to execute the cell below.
If running in Colab, you might see a warning about a non-existent
`__main__` module; this is expected behavior for a script-based
execution. To access the API, you would typically use a tool like
`curl` or a Python `requests` call from another cell or a local
machine. For example, to test from another Python cell:

```python
import requests

# Assuming Flask is running on default port 5000
login_url = 'http://127.0.0.1:5000/login'

# Successful login attempt
response_success = requests.post(login_url, json={'username': 'alice',
'password': 'alice_pass'})
print('Alice Login:', response_success.json())

# Failed login attempt (wrong password)
response_fail_pass = requests.post(login_url, json={'username': 'bob',
'password': 'wrong_pass'})
print('Bob Failed Login (password):', response_fail_pass.json())

# Failed login attempt (user not found)
response_fail_user = requests.post(login_url, json={'username':
'charlie', 'password': 'charlie_pass'})
print('Charlie Failed Login (user):', response_fail_user.json())
```

To stop the Flask app, you'll need to interrupt the execution of the
cell where it's running (e.g., by clicking the stop button next to the
cell output).
```

To run this Flask application, you'll need to execute the
Colab, you might see a warning about a non-existent `__main__` module; this is expected behavior for a script-based execution. To access the API, you would typically use a tool like `curl` or a Python `requests` call from another cell or a local machine. For example, to test from another Python cell:

```
import requests

# Assuming Flask is running on default port 5000
login_url = 'http://127.0.0.1:5000/login'

# Successful login attempt
response_success = requests.post(login_url, json={'username': 'alice', '
print('Alice Login:', response_success.json())

# Failed login attempt (wrong password)
response_fail_pass = requests.post(login_url, json={'username': 'bob', '
print('Bob Failed Login (password):', response_fail_pass.json())

# Failed login attempt (user not found)
response_fail_user = requests.post(login_url, json={'username': 'charlie
print('Charlie Failed Login (user):', response_fail_user.json())
```

To stop the Flask app, you'll need to interrupt the execution of the cell where it's running (e.g., by clicking the stop button next to the cell output).

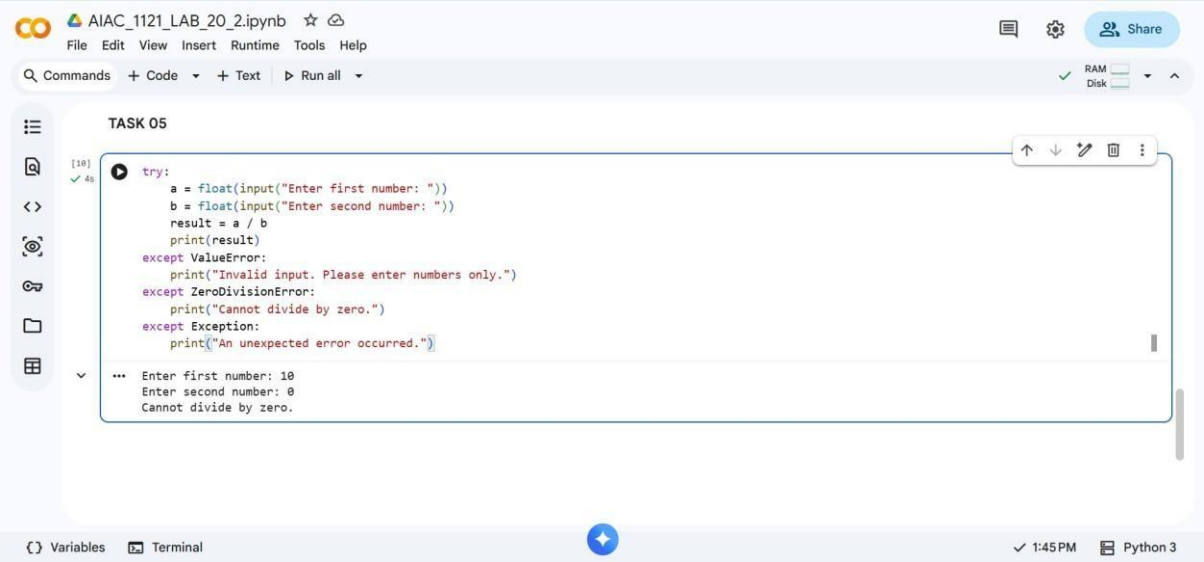
Explanation :

Common vulnerabilities include plain-text passwords, no validation, and insecure responses. bcrypt ensures passwords are securely stored and verified. Input is handled safely using JSON parsing. The API now reduces authentication risks and improves overall security.

Task – 05 : Exception Handling Improvement.

Prompt : Generate a Python program that takes two numbers and performs division.

Code & Output :



Explanation :

Programs without exception handling may crash during runtime errors. Invalid inputs and division by zero are common issues. try-except blocks ensure smooth execution and prevent crashes. Users receive clear error messages instead of program failure.

THANK YOU!!